Lab 3: Implementing and Comparing Search Algorithms in C++

Name: _____
Date: _____
Class/Section: _____

Objective:

The purpose of this lab is to implement sequential and binary search algorithms using object-oriented programming in C++. The lab aims to:

- Understand the differences between sequential and binary search algorithms.
- Apply both algorithms on arrays and vectors.
- Measure and compare the execution time of these search algorithms on different data structures.
- Analyze which search method is more efficient for specific scenarios.

Task 1: Sequential Search Implementation

1.1 Sequential Search for Arrays

- **Code Implementation:**

```cpp
// Sequential Search for Arrays
#include <iostream>
#include <chrono>

class SequentialSearch {
public:
    bool searchInArray(int arr[], int size, int target) {
        // TODO: Implement sequential search on arrays
    }
};
```

- **Execution Time Measurement:**

```cpp
// Fill in code to measure time and call the searchInArray function.
```

- **Results:**

Input Size	Time Taken (seconds)
100	_____
1000	_____
10,000	_____

1.2 Sequential Search for Vectors

- **Code Implementation:**

```
// Sequential Search for Vectors
#include <iostream>
#include <vector>
#include <chrono>

class SequentialSearch {
public:
    bool searchInVector(const std::vector<int>& vec, int target) {
        // TODO: Implement sequential search on vectors
    }
};
```

- **Execution Time Measurement:**

```
// Fill in code to measure time and call the searchInVector function.
```

- **Results:**

Input Size	Time Taken (seconds)
100	_____
1000	_____
10,000	_____

- **Comparison: Sequential Search (Array vs. Vector)**

Provide a brief analysis of the time comparison between the array and vector implementations of sequential search. Which one performed better, and why?

Task 2: Binary Search Implementation

2.1 Binary Search for Arrays

- **Code Implementation:**

```cpp
// Binary Search for Arrays
#include <iostream>
#include <chrono>

class BinarySearch {
public:
    bool searchInArray(int arr[], int size, int target) {
        // TODO: Implement binary search on arrays
    }
};
```

- **Execution Time Measurement:**

```cpp
// Fill in code to measure time and call the searchInArray function.
```

- **Results:**

Input Size	Time Taken (seconds)
100	_____
1000	_____
10,000	_____

2.2 Binary Search for Vectors:

- **Code Implementation:**

```cpp
// Binary Search for Vectors
#include <iostream>
#include <vector>
#include <chrono>

class BinarySearch {
public:
    bool searchInVector(const std::vector<int>& vec, int target) {
        // TODO: Implement binary search on vectors
    }
};
```

- **Execution Time Measurement:**

```
// Fill in code to measure time and call the searchInVector function.
```

- **Results:**

Input Size	Time Taken (seconds)
100	_____
1000	_____
10,000	_____

Comparison: Binary Search (Array vs. Vector)

Provide a brief analysis of the time comparison between the array and vector implementations of binary search. Which one performed better, and why?

Overall Comparison: Sequential vs. Binary Search

Summarize the key differences between sequential and binary search based on their performance. Which algorithm performed better for large datasets? How does the choice of array vs. vector impact the performance?

Submission Guidelines:

- Submit this completed lab report with all required answers and comparisons.
- Attach the source code files (.cpp) for your search implementations, properly commented for clarity.